D20A / 62

Experiment No: 4

AIM: To develop a blockchain-powered web application using Solidity programming language on Remix IDE and embedding Metamask Wallet

THEORY:

A **blockchain** is a decentralised, append-only digital ledger whose records are replicated across many nodes and cannot be changed without being noticed. This experiment uses that idea to build a **Digital Warranty and Product Ownership Transfer System**, where the warranty and the owner of a product are recorded on-chain instead of on paper.

The application is a **Solidity smart contract** written in **Remix IDE** and deployed on the **Ethereum Sepolia Testnet**. **MetaMask** works as the wallet: it links the user's account to Remix and asks for approval, paying the gas fee in test ETH, each time a transaction is sent.

For every registered product, the contract keeps the following details in a struct:

- Unique product ID
- Product name
- Current owner's wallet address
- Warranty duration
- Purchase timestamp
- Existence flag (true once registered)

Four public functions operate on this data: one registers a new product, one tells whether the warranty is still valid, one reads back the stored record, and one hands the product over to a new owner.

The main blockchain concept shown here is **rule enforcement by the smart contract itself**. The transferOwnership() function begins with a check that lets only the present owner proceed:

```
require(
   products[_productId].owner == msg.sender,
   "Only current owner can transfer"
);
```

Once ownership moved from Account 1 to Account 2, Account 1 was no longer the registered owner, so its next transfer attempt was reverted by the contract with the message:

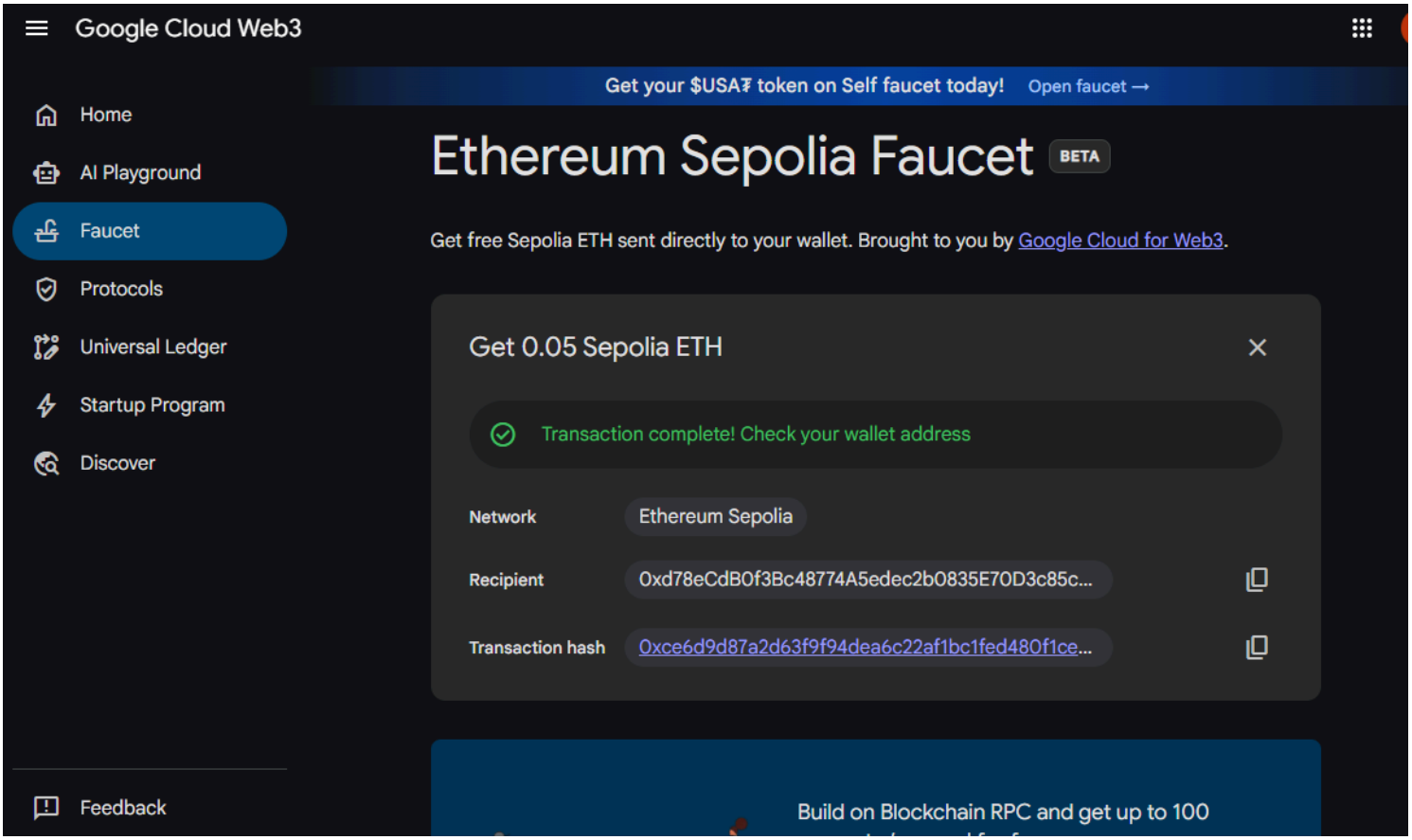
Only current owner can transfer
Thus the rule is enforced automatically by the contract code, with no third party involved.

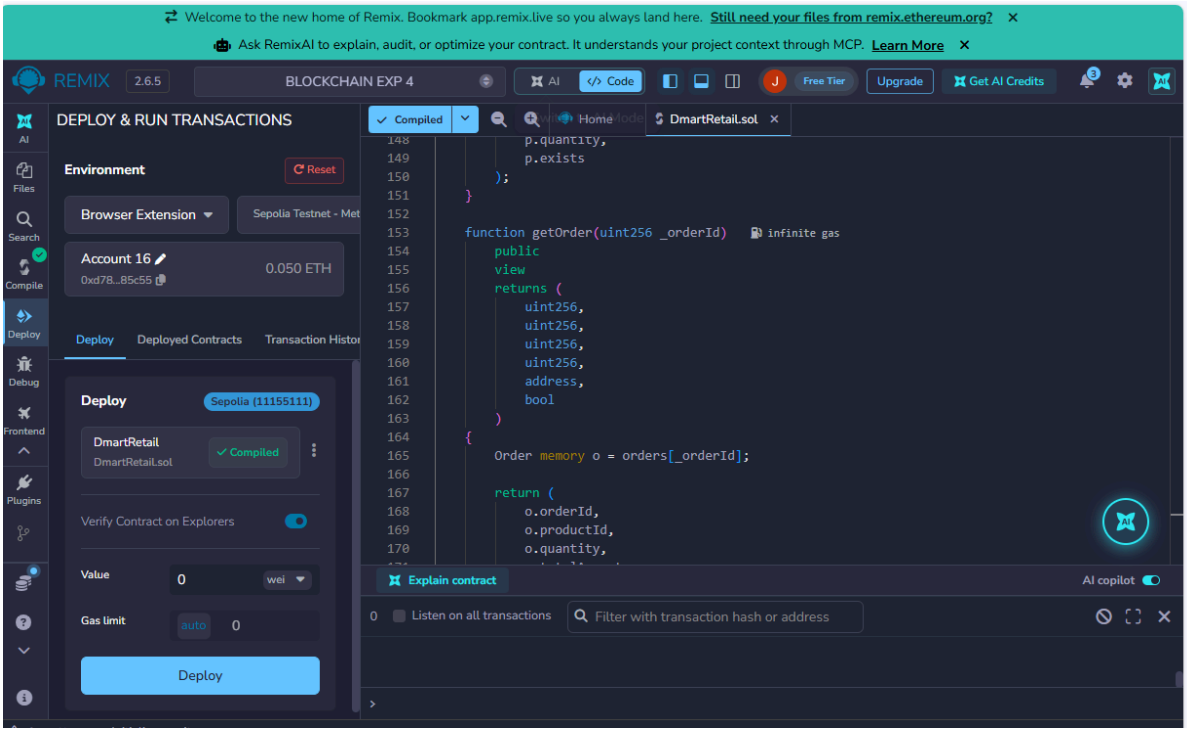
The workflow follows the lab procedure: preparing MetaMask and linking the Sepolia testnet to

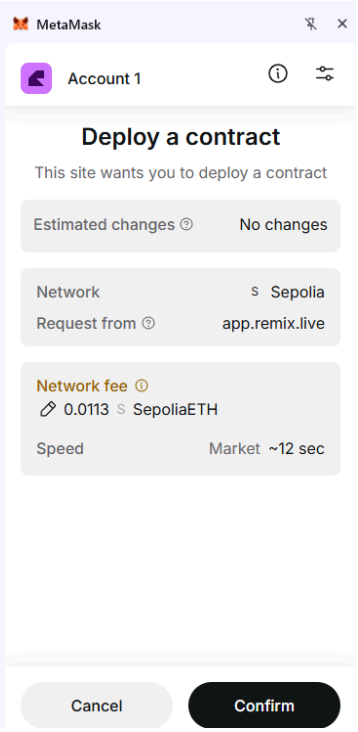
Remix, writing and deploying the contract, inspecting the deployment transaction on the block explorer, and then calling the functions of the deployed contract.

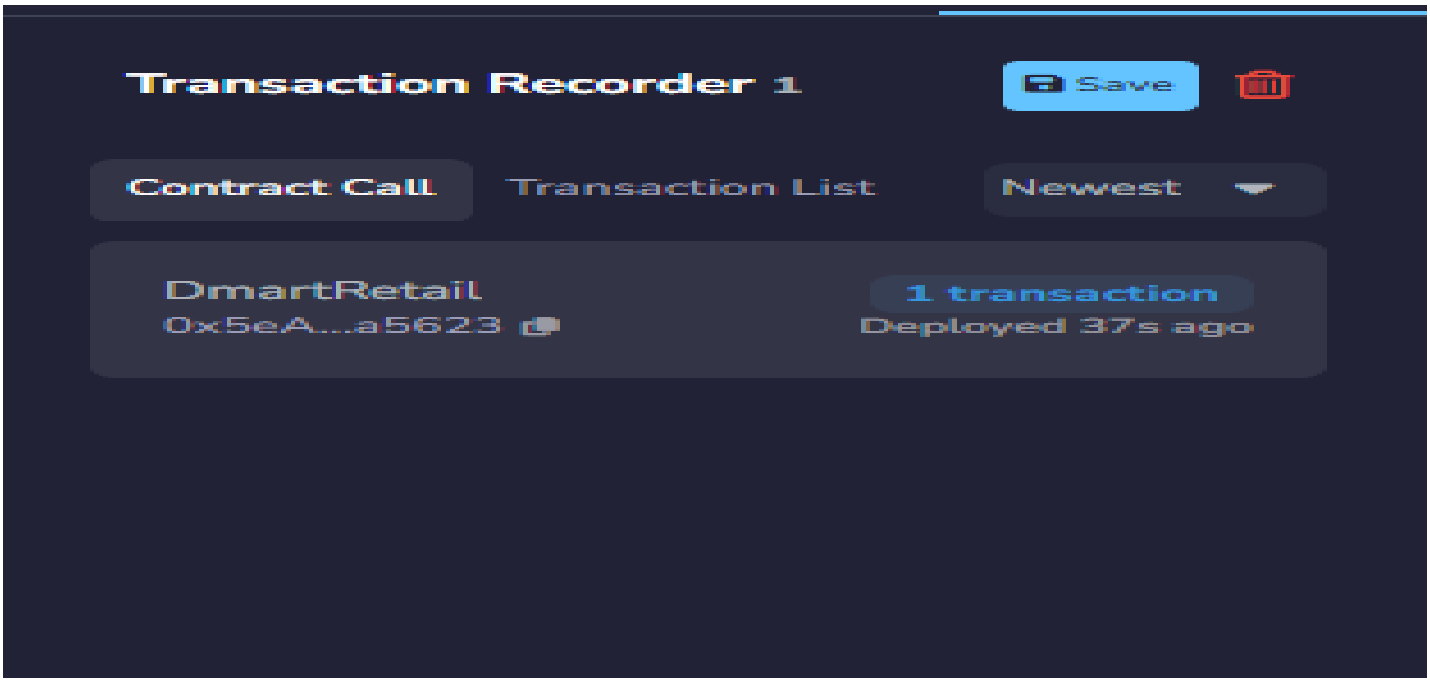
STEPS and OUTPUT:

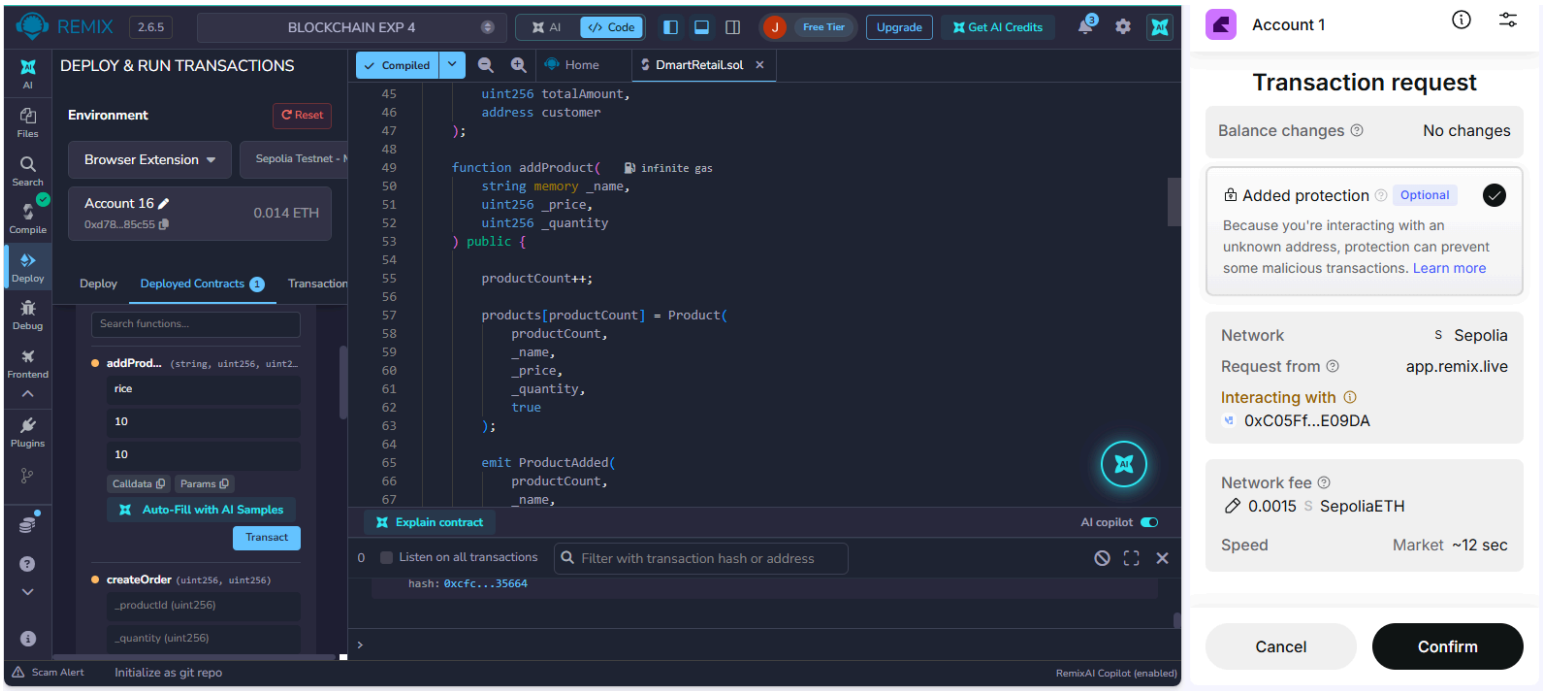
1. Install MetaMask, switch to Sepolia and connect it to Remix IDE
2. Write the DigitalWarranty smart contract in Solidity
3. Compile the contract and deploy it on Sepolia through MetaMask
4. Register a product by calling registerProduct()
5. Fetch the stored product details using getProduct()
6. Confirm that the warranty is active using checkWarranty()
7. Transfer the product to Account 2 with transferOwnership()
8. Call getProduct() again to confirm the new owner
9. Attempt a transfer from Account 1 to show that it is rejected

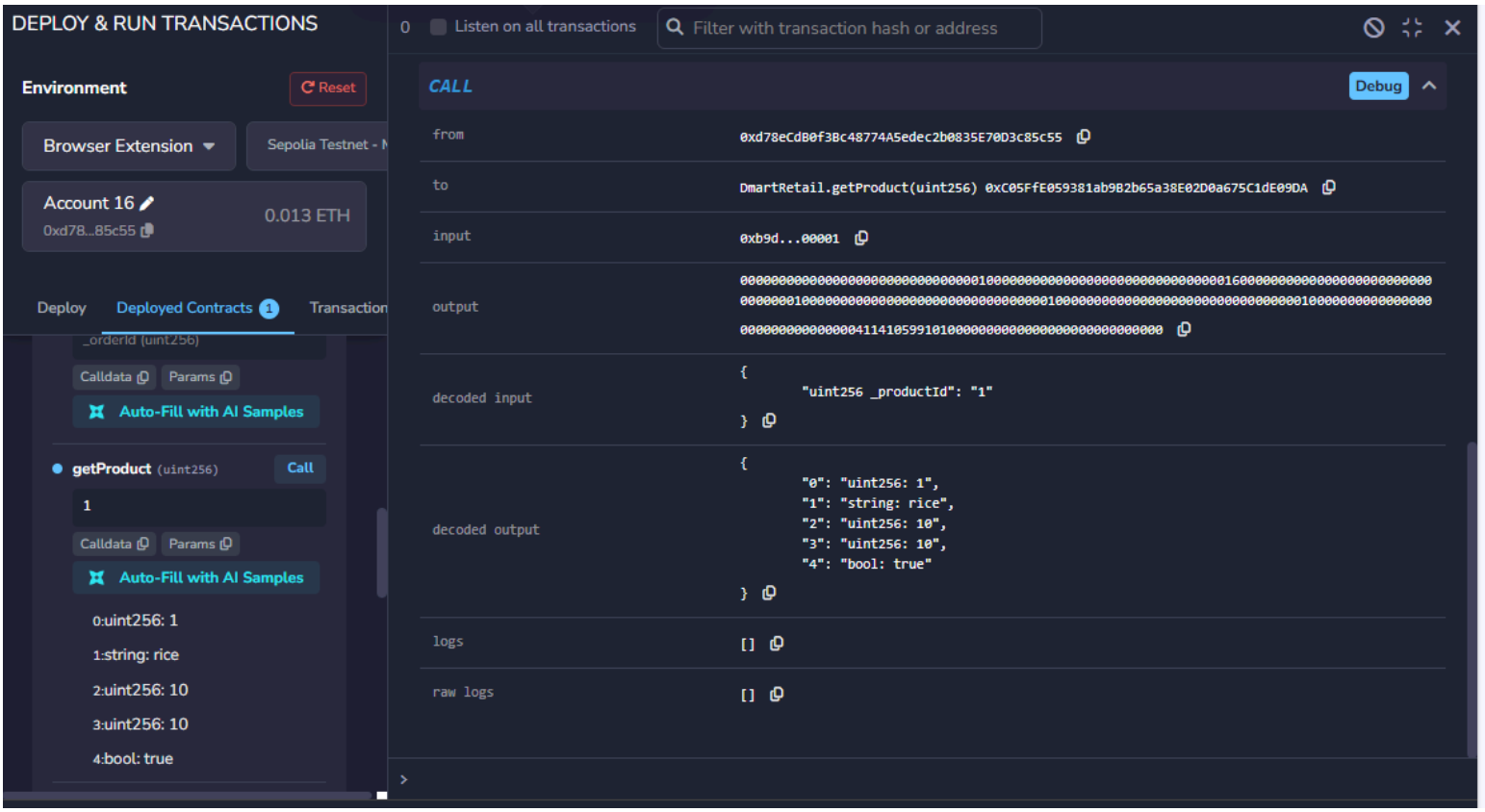


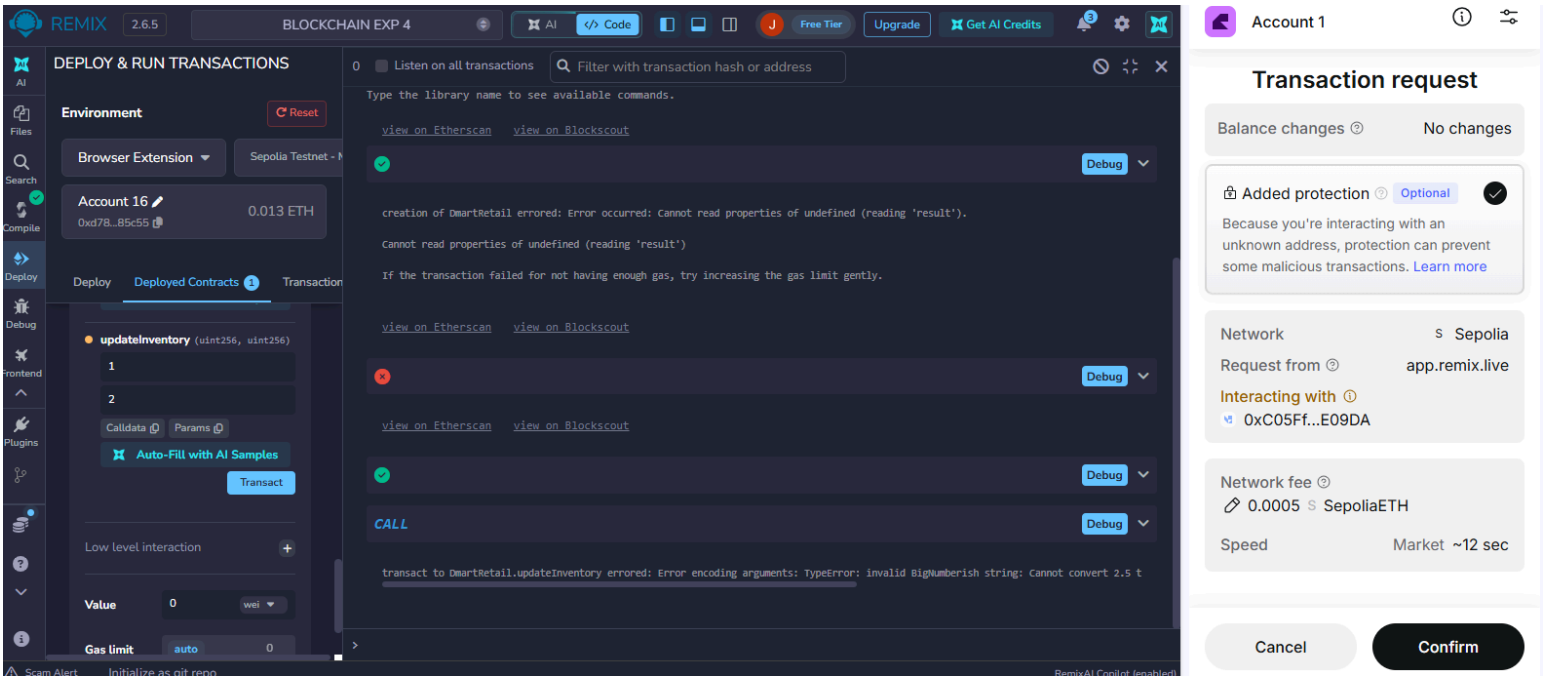

Welcome to the new home of Remix. Bookmark app.remix.live so you always land here. Still need your files from remix.ethereum.org?
Ask RemixAI to explain, audit, or optimize your contract. It understands your project context through MCP. Learn More
REMIX 2.6.5
BLOCKCHAIN EXP 4
Free Tier
Upgrade
Get AI Credits
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
Sepolia Testnet - Met
Account 16
0xd78...85c55
0.050 ETH
Deploy
Deployed Contracts
Transaction Histor
Deploy
Sepolia (11155111)
DmartRetail
DmartRetail.sol
Compiled
Verify Contract on Explorers
Value
0
wei
Gas limit
auto
0
Deploy
Compiled
Home
DmartRetail.sol
p.quantity,
p.exists
);
}
function getOrder(uint256 _orderId) infinite gas
public
view
returns (
uint256,
uint256,
uint256,
uint256,
address,
bool
)
{
Order memory o = orders[_orderId];
return (
o.orderId,
o.productId,
o.quantity,
Explain contract
AI copilot
0
Listen on all transactions
Filter with transaction hash or address

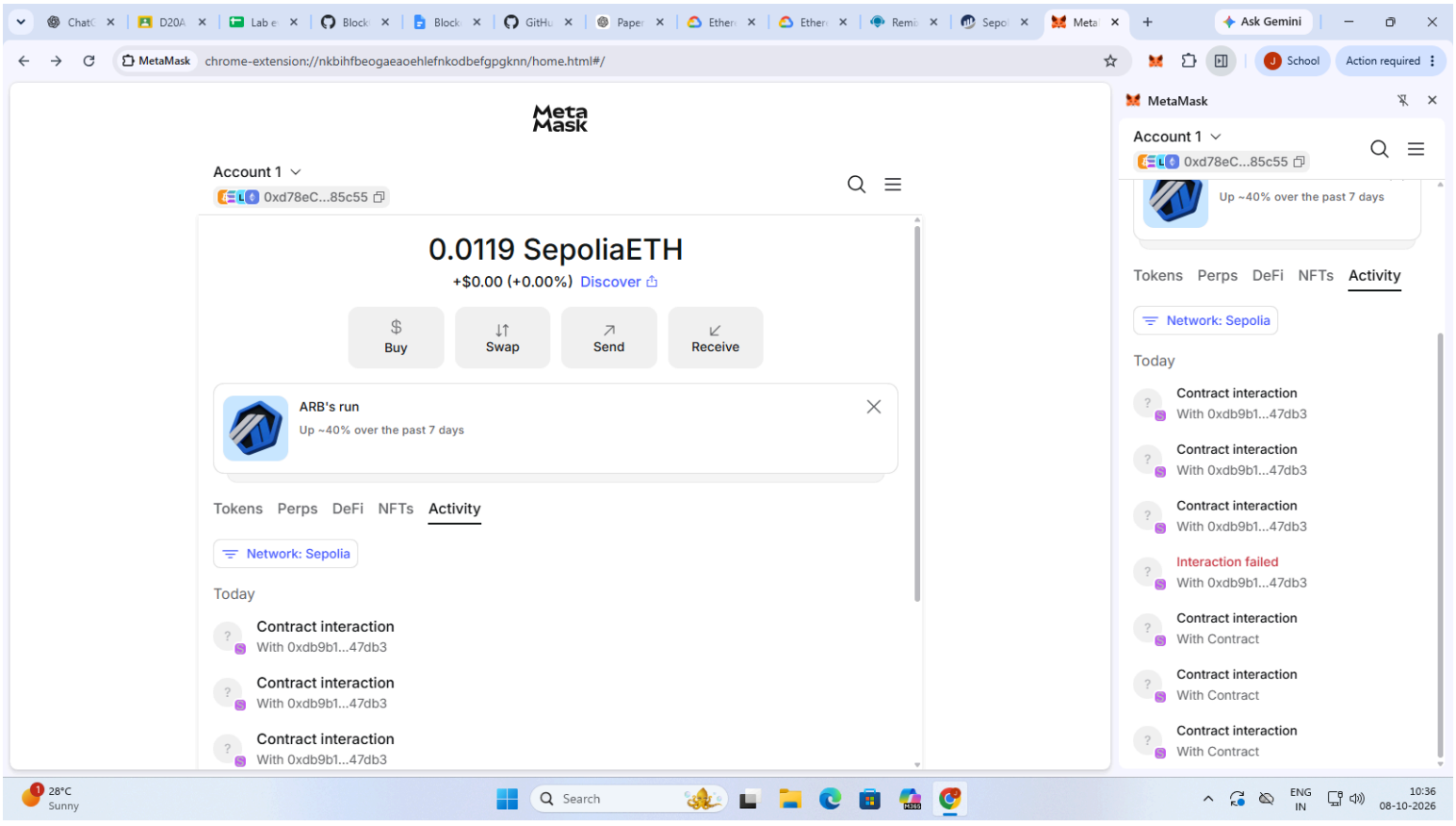

MetaMask
Account 1
Deploy a contract
This site wants you to deploy a contract
Estimated changes
No changes
Network
Sepolia
Request from
app.remix.live
Network fee
0.0113 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm


Transaction Recorder 1
Save
Contract Call
Transaction List
Newest
DmartRetail
0x5eA...a5623
1 transaction
Deployed 37s ago


REMIX 2.6.5
BLOCKCHAIN EXP 4
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
Sepolia Testnet
Account 16
0xd78...85c55
0.014 ETH
Deploy
Deployed Contracts 1
Transaction
Search functions...
addProd... (string, uint256, uint2...
rice
10
10
Calldata
Params
Auto-Fill with AI Samples
Transact
createOrder (uint256, uint256)
_productId (uint256)
_quantity (uint256)
Compiled
Home
DmartRetail.sol
uint256 totalAmount,
address customer
);
function addProduct(infinite gas
string memory _name,
uint256 _price,
uint256 _quantity
) public {
productCount++;
products[productCount] = Product(
productCount,
_name,
_price,
_quantity,
true
);
emit ProductAdded(
productCount,
_name,
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or address
hash: 0xcfc...35664
Scam Alert
Initialize as git repo
RemixAI Copilot (enabled)
Account 1
Transaction request
Balance changes
No changes
Added protection
Optional
Because you're interacting with an unknown address, protection can prevent some malicious transactions. Learn more
Network
Sepolia
Request from
app.remix.live
Interacting with
0xC05Ff...E09DA
Network fee
0.0015 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm


DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
Sepolia Testnet
Account 16
0xd78...85c55
0.013 ETH
Deploy
Deployed Contracts 1
Transaction
_orderId (uint256)
Calldata
Params
Auto-Fill with AI Samples
getProduct (uint256)
Call
1
Calldata
Params
Auto-Fill with AI Samples
0:uint256: 1
1:string: rice
2:uint256: 10
3:uint256: 10
4:bool: true
0
Listen on all transactions
Filter with transaction hash or address
CALL
Debug
from
0xd78eCdB0f3Bc48774A5edec2b0835E70D3c85c55
to
DmartRetail.getProduct(uint256) 0xC05FfE059381ab9B2b65a38E02D0a675C1dE09DA
input
0xb9d...00001
output
0000000000000000000000000000000000000000000000000000000000000020
0000000000000000000000000000000000000000000000000000000000000001
0000000000000000000000000000000000000000000000000000000000000020
0000000000000000000000000000000000000000000000000000000000000004
decoded input
{
"uint256 _productId": "1"
}
decoded output
{
"0": "uint256: 1",
"1": "string: rice",
"2": "uint256: 10",
"3": "uint256: 10",
"4": "bool: true"
}
logs
[]
raw logs
[]


REMIX 2.6.5
BLOCKCHAIN EXP 4
DEPLOY & RUN TRANSACTIONS
Environment
Reset
Browser Extension
Account 16
0xd78...85c55
0.013 ETH
Deploy
Deployed Contracts
Transaction
updateInventory (uint256, uint256)
1
2
Calldata
Params
Auto-Fill with AI Samples
Transact
Low level interaction
Value
0
wei
Gas limit
auto
0
Listen on all transactions
Filter with transaction hash or address
Type the library name to see available commands.
view on Etherscan
view on Blockscout
Debug
creation of DmartRetail errored: Error occurred: Cannot read properties of undefined (reading 'result').
Cannot read properties of undefined (reading 'result')
If the transaction failed for not having enough gas, try increasing the gas limit gently.
CALL
transact to DmartRetail.updateInventory errored: Error encoding arguments: TypeError: invalid BigNumberish string: Cannot convert 2.5 t
Account 1
Transaction request
Balance changes
No changes
Added protection
Optional
Because you're interacting with an unknown address, protection can prevent some malicious transactions. Learn more
Network
Sepolia
Request from
app.remix.live
Interacting with
0xC05Ff...E09DA
Network fee
0.0005 SepoliaETH
Speed
Market ~12 sec
Cancel
Confirm


MetaMask
chrome-extension://nkbihfbeogaeaoehlefnkodbefgpgknn/home.html#/
Account 1
0xd78eC...85c55
0.0119 SepoliaETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive
ARB's run
Up ~40% over the past 7 days
Tokens
Perps
DeFi
NFTs
Activity
Network: Sepolia
Today
Contract interaction
With 0xdb9b1...47db3
Contract interaction
With 0xdb9b1...47db3
Contract interaction
With 0xdb9b1...47db3
Interaction failed
With 0xdb9b1...47db3
Contract interaction
With Contract
Contract interaction
With Contract
Contract interaction
With Contract

Conclusion:

The blockchain-powered Digital Warranty and Product Ownership Transfer System was successfully implemented using Solidity and Remix IDE. The smart contract was deployed on the Sepolia Testnet using MetaMask. A product was registered with its warranty details, the warranty status was verified, and ownership was successfully transferred to another blockchain account. The updated ownership and active warranty were verified through the deployed smart contract.